Big Data - Bachelor's Degree in Data Science and Engineering

Universidad de Las Palmas de Gran Canaria

Academic Year 2026-2027

Search Engine Project: Stage 2

Building a Service-Oriented Architecture

Group Name: Query Tarantino

GitHub Repository:

`https://github.com/Query-Tarantino/stage_2`

Team Members:

Abel Castro Villar	ID: 78593645T
Hugo González Fernández	ID: 54587350R
Javier Martín Rodríguez	ID: 43386897L
Máximo Prosperi Reynoso	ID: 42252471E

November 2026

Contents

1 Introduction and Objectives **3**

2 System Architecture **4**
2.1 Services and shared storage 4
2.2 The life of a book 4
2.3 Processes, replicas and threads 5
2.4 Coordination mechanism 6

3 API Design and Contracts **8**
3.1 Front-end interface 8
3.2 Operational interface and completion semantics 9
3.3 Internal contracts 9
3.4 Errors 10
3.5 Validation evidence 10

4 Microservice Implementation **11**
4.1 Code structure and technologies 11
4.2 Shared contracts in `common` 12
4.3 Conditional writes and leases 12
4.4 The indexer 13
4.5 Crawler, query and control 14
4.6 Coordination modes in code 14
4.7 Deployment and startup 15
4.8 Testing and continuous integration 15

5 Design Decisions **16**
5.1 Coordination mechanism 16
5.2 The five modes measured, and the winner 16
5.3 Single-machine deployment 18
5.4 API design 18
5.5 Storage 18
5.6 Concurrency model and configuration 19
5.7 Smaller decisions 19

6 Benchmarks and Results **20**
6.1 Setup and methodology 20
6.2 Microbenchmarks 20
6.3 Ingestion throughput 21
6.4 Indexing throughput and latency 22
6.5 Search latency under concurrency 22
6.6 Coordination cost 23
6.7 Resource usage 24
6.8 Coordination mechanisms compared 24
6.9 Recovery 26
6.10 Bottlenecks and the limits of this evidence 27

7 Conclusions and Future Improvements **27**
7.1 What the measurements say 27
7.2 From one host to the Stage 3 cluster 28
7.3 The winning mode in Stage 3 29
7.4 Lessons 29

8 Statement on AI Usage **29**

References **30**

A Reproducibility **31**

B Endpoint Inventory **32**

C Failure Matrix **33**

D Configuration **34**

1 Introduction and Objectives

Stage 1 [2] built the data layer of our search engine over Project Gutenberg: a datalake with the `batch` layout and an inverted index in MongoDB, chosen after benchmarking three languages and several structures. Stage 2 [1] turns that data layer into cooperating services. Each service runs as its own process, they share explicit contracts, and an explicit mechanism coordinates them. Everything runs on one machine, and replicas and threads are part of the design from the start. The goal is a system whose limits we can measure on that machine and whose design we can carry to the Stage 3 cluster.

We set six objectives and met all of them:

1. **Four services and one module of contracts.** `crawler` (Ingestion), `indexer` (Indexing), `query` (Search) and `control` (seeding and reconciliation) are written in Java 25 with Javalin and Jackson. They share `common`, which is the only place where the code names a collection, a message, a state or a variable. MongoDB is the source of truth, the datalake is a Docker volume, and nginx is the only entry point.
2. **An explicit, failure-proof coordination mechanism.** Work travels as messages on Hazelcast queues, MongoDB decides who owns each job, and `control` periodically repairs whatever a crash leaves behind. A book becomes searchable only through a published batch, and it never appears in two published batches of the same generation.
3. **Documented and validated APIs.** The front-end, operational, health and internal interfaces are kept as separate inventories in `api/openapi.yaml` (OpenAPI 3.1, version 1.10). The non-HTTP contracts are in `SPEC.md`. An automated checker validates every response against the specification in continuous integration, in every coordination mode.
4. **Recovery from every failure we could cause.** The failure matrix has 14 rows, each with its own script. All 165 recovery runs of the official campaign recover, and every row gives the same books and search answers with 1, 2 and 3 replicas and in every mode where it applies.
5. **Alternatives measured, not only argued.** The same system can coordinate in four other ways behind one variable: ActiveMQ with and without persistence, polling of shared state, and classical orchestration. One campaign on one machine compared all five against each other.
6. **Benchmarks that can be traced.** JMH microbenchmarks and nine integration experiments ran with five interleaved repetitions and 95 % confidence intervals. Every figure in this report names the run it comes from, and that run keeps its command, commit, configuration and raw data in the repository (Appendix A).

The campaign answers the questions of the assignment. Indexing limits the pipeline: one indexer publishes 28.5 books/s, while one crawler downloads about 500. One `query` replica saturates at about 4,400 requests/s. Coordination itself is cheap in every mode, below 17 % of one core. The default mode, `hazelcast`, remains the winner under the decision rule we fixed before the campaign (Section 5.2).

Table 1 shows how the team split the work. Each member owned one piece and reviewed another member's pull requests, and changes to the shared contracts required all four. The work advanced through seven phases, F0 to F6, and each phase had a gate: the walking skeleton had to show a new book in `/search`, and F3 required the whole failure matrix to pass. The history of `dev` shows the contribution of each member.

Table 1: Pieces owned by each member. Each owner wrote the code, scripts and runs of their piece.

Member	Piece
Hugo González Fernández (A)	`crawler` (Ingestion), Docker Compose and the shared `Dockerfile`; `ingest-throughput`
Máximo Prosperi Reynoso (B)	`indexer` (Indexing and batch publication); `index-throughput`; the four alternative coordination modes, their experiments and the campaign script
Abel Castro Villar (C)	`query` (Search), nginx, `api/openapi.yaml` and the `checks` module; the load generator and `search-latency`
Javier Martín Rodríguez (D)	`control` (seeding and reconciliation), the failure matrix and its 14 recovery scripts, result processing in `bench`; `coordination-cost`, `resource-usage`, and the official campaign on his machine

2 System Architecture

The system has three core services and one control process, all on a single host. They never call each other in the default mode: they exchange messages through Hazelcast and keep their data in the datalake and in MongoDB. Figure 1 shows the whole deployment: the front-end boundary, the internal interactions, the replicas and threads of each process, and the shared storage.

2.1 Services and shared storage

- **`crawler` (Ingestion Service)** takes `BookTask` messages from `books-to-download`. It claims each book in MongoDB, downloads it from Gutenberg or from the local mirror and stores its header and body in the datalake with the Stage 1 `batch` layout. It marks the book `DOWNLOADED` and then enqueues it in `books-to-index`.
- **`indexer` (Indexing Service)** groups `books-to-index` into batches of up to 100 books or 30 s. It extracts the metadata of each book and counts its terms. It writes metadata and postings into the collections of the write generation and publishes the batch with one conditional write. Then it announces `BatchPublished`.
- **`query` (Search Service)** keeps an immutable snapshot of the published batches of the current generation. It serves keyword search with author, language and year filters, TF-IDF ranking, book details and filter values.
- **`control`** seeds the candidate books at startup and reconciles every 30 s. It releases expired leases, abandons expired batches, requeues lost messages, drives index rebuilds and deletes orphan `.tmp` files. In the default mode it never drives a book step by step, so it is not an orchestrator.
- **MongoDB** is the only source of truth. It holds `book_states`, one document per book, and `index_state`, which points to the generation `query` serves. Each generation of the index has three collections, `metadata_g<n>`, `postings_g<n>` and `batches_g<n>`.
- **Hazelcast** carries two work queues and one topic. It is not a source of truth: a lost or repeated message never corrupts state, because the claim in MongoDB decides who owns each book.

2.2 The life of a book

Figure 2 shows the states of a book and of a batch, and who writes each transition. Every arrow is one conditional write to one MongoDB document (Section 4.3). A book becomes eligible for indexing only once its header and body are in the datalake, and it becomes searchable only when the batch that holds it is `PUBLISHED` in the current generation and `query` has refreshed its snapshot.

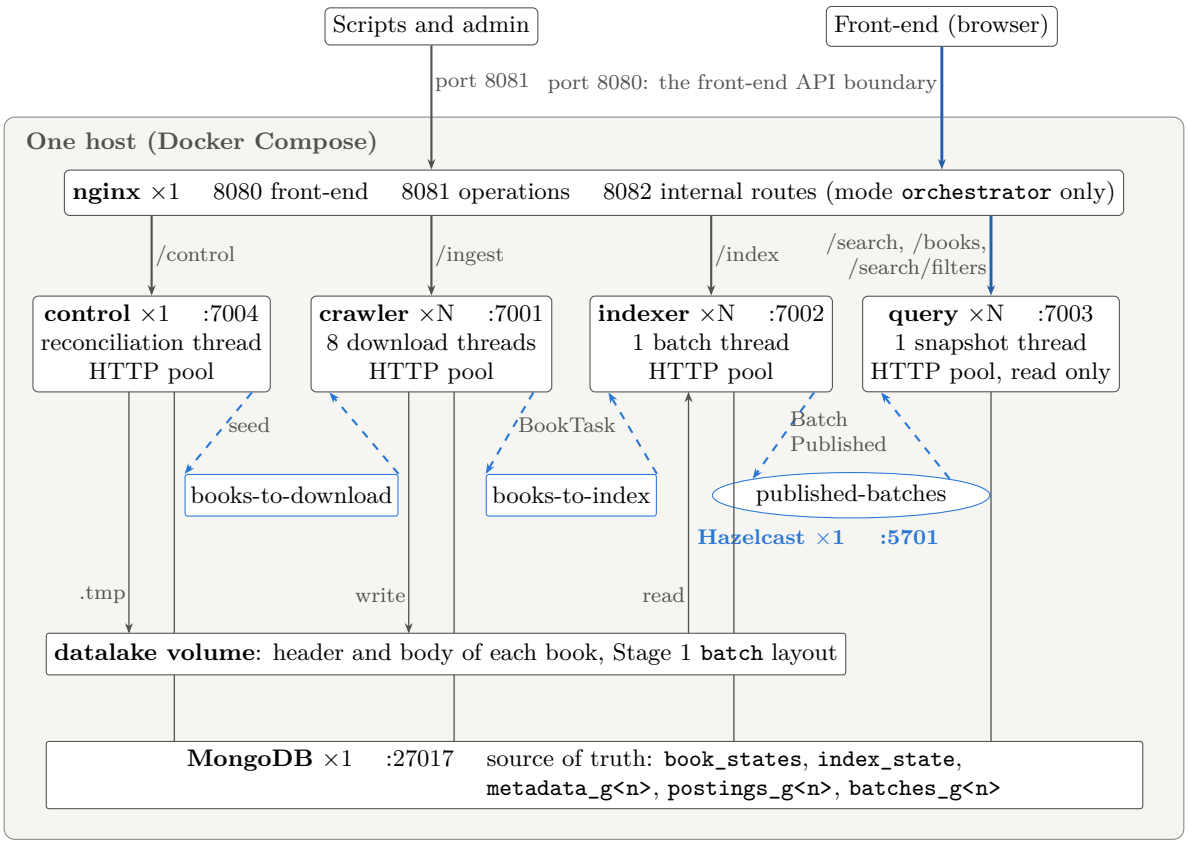


Figure 1: Deployment on the single Stage 2 host, in the default mode `hazelcast`. Thick blue arrows are front-end requests through nginx port 8080, gray arrows are operational requests through port 8081, dashed arrows are Hazelcast messages and thin vertical lines are storage access; every service reads and writes MongoDB. "×N" marks a service that scales with `docker compose up --scale`, and each box lists the threads of one replica. Not drawn: `crawler` also re-enqueues retried downloads in `books-to-download`, and `control` re-enqueues lost messages and reindexing in both queues (`SPEC.md` §4.2).

2.3 Processes, replicas and threads

A service is a logical responsibility, and each replica is a separate JVM process. Table 2 lists the replicas and threads of each process. Request handlers never run long jobs: they only validate, write state and enqueue, so the HTTP pool only waits on MongoDB and the broker. Within a process, threads share only an immutable snapshot or a `java.util.concurrent` structure, and the code takes no lock of its own. Replicas share nothing in memory: they meet only in MongoDB, through conditional writes, and in the broker.

The design keeps replicas and threads from interfering in four ways:

- **Duplicate work.** A message, an HTTP call or a poll is only a hint. The claim in MongoDB lets one worker take a book or a batch, and a duplicate fails its claim and is dropped.
- **Conflicting writes.** Every state change is one conditional single-document write whose filter carries the expected state and the `claim_id`. A worker that loses its lease writes nothing else, and a write that does not match is never retried.
- **Thread exhaustion.** Handlers answer in milliseconds and never wait for a job. Worker threads are a fixed number per replica, and a JVM that runs out of memory exits instead of losing one thread.
- **Contention.** Replicas compete for the single MongoDB, the broker and the datalake volume. Batches share a per-batch cost among their books, postings go in `bulkWrite`s of 1,000 terms, and `query` reads an immutable snapshot without locks. Section 6 measures the contention that is left.

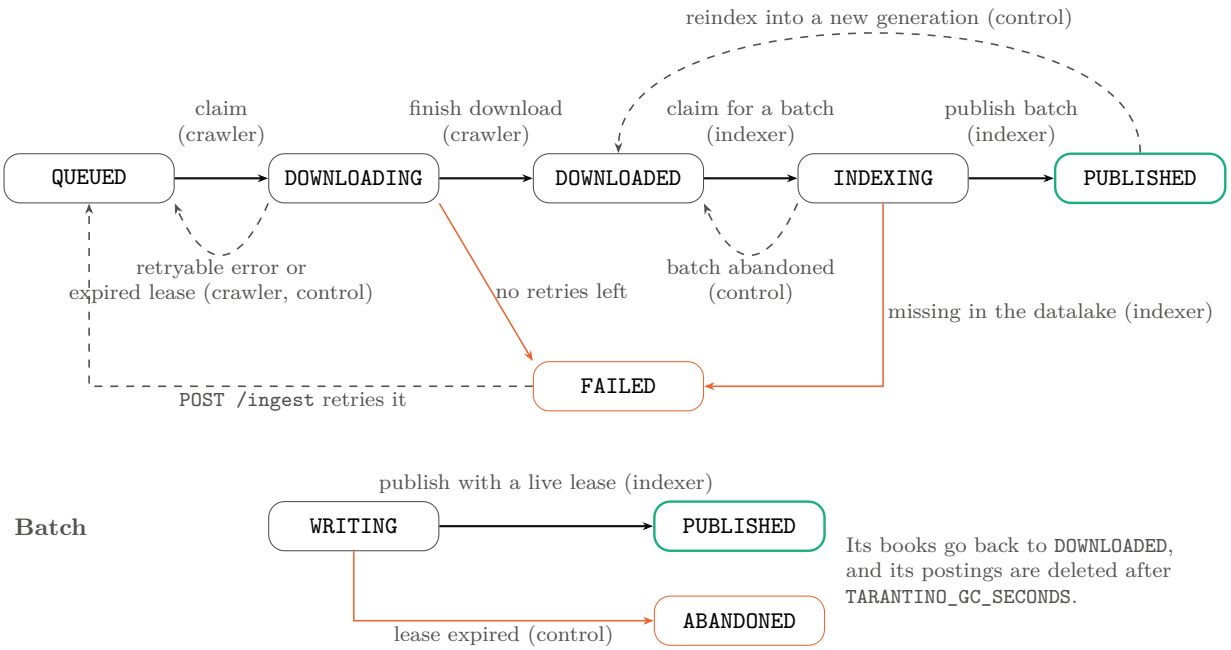


Figure 2: States of a book (`book_states.status`) and of a batch (`batches_g<n>.status`), with the service that writes each transition. Solid arrows are the normal path; dashed arrows are repairs. A `PUBLISHED` batch never changes again, and an `ABANDONED` one is never seen by `query`.

2.4 Coordination mechanism

The coordination is a hybrid of two mechanisms of the assignment, choreography with a broker and time-based coordination, over one arbiter, MongoDB (Table 3). The same pattern appears twice: an event for speed and a period in case the event is lost.

The same code runs in four other modes, selected with `TARANTINO_COORDINATION` and one Compose override per mode (Table 4). Only two things change between modes: how work reaches the workers and how `query` learns of a batch. The datalake, MongoDB, the conditional writes, the index, the search and the front-end API stay the same, so the modes can be compared on equal terms (Section 6.8).

Table 2: Replicas and threads per process (`SPEC.md` §2.2). Javalin's HTTP pool grows from 8 to 250 threads unless `TARANTINO_HTTP_THREADS` caps it.

Process	Replicas	Threads
`crawler`	N, 1 by default	`TARANTINO_PARALLEL_DOWNLOADS` download workers (8 by default), one lease heartbeat per book on a virtual thread, HTTP pool
`indexer`	N, 1 by default	One batch loop (`index-batches`), one lease heartbeat per open batch on a virtual thread, HTTP pool
`query`	N, 1 by default	One snapshot thread, the only one that refreshes; the topic listener only asks it to; HTTP pool that only reads the snapshot
`control`	Always 1	One `reconciliation` thread for reconciliation and datalake cleanup, so they never overlap; one `orchestrator` thread in that mode; HTTP pool
`mongo`, `hazelcast`, `nginx`	1 each	Their own; nginx runs one worker per CPU

Table 3: The three parts of the default coordination and the responsibility of each.

Mechanism	Where	Responsibility
Choreography with a broker	`books-to-download` and `books-to-index` (Hazelcast `IQueue`); `published-batches` (`ITopic`)	Normal path: a free worker takes a book as soon as it is queued; `query` refreshes on each `BatchPublished`
Time-based coordination	`control` reconciles every 30 s; `query` refreshes every 5 s	Safety net: expired leases, abandoned batches, lost messages and lost announcements, with `book_states` as the persistent marker
Shared persistent state	MongoDB conditional writes (`SPEC.md` §5.1)	Arbiter: the claim acknowledges the message, so a duplicate fails and is dropped

Table 4: Coordination modes (`SPEC.md` §11.1). `activemq-persistent` is `activemq` with persistent delivery and acknowledgement after the job.

Mode	Mechanism	Workers take work from	`query` learns of a batch	Lost work comes back by
`hazelcast`	Choreography over Hazelcast	`IQueue`	Topic listener and period	`control` requeue
`activemq`	The same over ActiveMQ Classic 6	JMS queues	JMS topic and period	`control` requeue; broker redelivery with persistence
`polling`	Time-based, over shared state	A sorted claim in MongoDB	Period only	Nothing to requeue: waiting books are polled
`orchestrator`	Classical orchestration by `control`	`POST /internal/...` into a local queue	`control`'s call and period	`control` dispatching again

3 API Design and Contracts

The API has four separate inventories: front-end, operational, health and internal. Each one has a different consumer and a different nginx port (Table 5). The full specification is `api/openapi.yaml`, in OpenAPI 3.1 [4]. Its `info.version` follows the version of `SPEC.md`, now 1.10, and its tags are the four inventories. Appendix B lists every operation with its responses.

Table 5: Endpoint inventories. The front-end only sees port 8080, and only `query` serves it.

Inventory	Operations	Provider	Consumer
Front-end, port 8080	`GET /search`, `GET /books/{book_id}`, `GET /search/filters`	`query` replicas behind nginx	Front-end, and the `checks` and the load generator acting as one
Operational, port 8081	`POST /ingest/{book_id}`, `GET /ingest/status/{book_id}`, `GET /ingest/list`	`crawler`	Admin and load scripts, `checks`
	`POST /index/update/{book_id}`, `GET /index/status/{book_id}`, `POST /index/rebuild`, `GET /index/status`	`indexer`	Admin, benchmark scripts
	`GET /control/status`	`control`	Admin, `bench wait`
Health, no nginx	`GET /health` on each service port	Every service	Compose healthchecks
Internal, port 8082	`POST /internal/download`, `POST /internal/index`, `POST /internal/snapshot/refresh`	`crawler`, `indexer`, each `query`	`control`, in mode `orchestrator` only

3.1 Front-end interface

The front-end needs three use cases: searching with optional filters and pages, showing one book, and offering the filter values. Search is one endpoint with optional parameters, not one route per filter.

```
GET /search?q=pride%20prejudice&author=austen&language=en&offset=0&limit=20
200 OK
{"query": "pride prejudice", "filters": {"author": "austen", "language": "en"},
 "total": 1, "offset": 0, "limit": 20,
 "results": [{"book_id": 1342, "title": "Pride and Prejudice", "author": "Jane Austen",
              "language": "en", "year": 1998}]}
```

Table 6 gives the rules of each parameter. Parameters are validated in the order of the table, and the first one that fails gives `400` with Problem Details naming it in `parameter`. A valid query that matches nothing answers `200` with `total: 0` and an empty list. `total` counts every match after the filters, while `results` holds only the requested page.

`GET /books/{book_id}` returns the fields of a search result plus every language and the original year. It answers `404` for a book that is not in the published snapshot, even if it is being indexed. `GET /search/filters` returns the languages with their book counts, the range of years and the number of searchable books, computed once per snapshot. All three routes answer `503` until `query` holds its first snapshot. `query` allows CORS from `TARANTINO_CORS_ORIGIN` (`*` by default), so a browser front-end served from another origin can call port 8080 directly.

Table 6: Parameters and semantics of `GET /search` (`SPEC.md` §6.3).

Parameter	Rule
`q`	Required. Tokenized with the same tokenizer as the index, and a book must contain every word. Missing or blank: `400 MISSING_PARAMETER`; only stopwords: `200` with no results
`author`	Every word of the author, through the same tokenizer, must be among the book's `author_terms`: `austen` finds "Jane Austen"
`language`	ISO 639-1 code, two lowercase letters. A book with several languages matches any of them; an unknown code gives an empty `200`
`year`	Four digits, equal to the Gutenberg release year of the book
`offset`, `limit`	0 and 20 by default; `offset` ≥ 0, `limit` from 1 to 100
Order	Descending TF-IDF, the sum of $(1 + \ln tf) \cdot \ln(N/df)$ over the query words; ties by ascending `book_id`

3.2 Operational interface and completion semantics

Every request that changes state answers `202 Accepted` with the book's current state and a `Location` header that points to it, or `200` if the work was already done. The caller polls that `Location`: an accepted book has finished when its state is `PUBLISHED`, and has failed when it is `FAILED`, with the reason in `last_error`. Handlers never download or index, so they answer in milliseconds, and a client can use a 5 s timeout. This replaces the synchronous `POST /ingest` of the assignment's example, which would hold a request thread for a whole download.

- `POST /ingest/{book_id}` creates the book's state and enqueues it. For a book that is already queued or downloading, it answers `202` without enqueuing it again; for a `FAILED` book, it retries it.
- `POST /index/update/{book_id}` enqueues a `DOWNLOADED` book. It answers `409 NOT_DOWNLOADED` for a book that is not in the datalake yet, `202` without a new message for one that is already `INDEXING`, and `200` for a `PUBLISHED` one.
- `POST /index/rebuild` opens a rebuild of the next generation, and `GET /index/status` reports its progress in `books_pending`. A second request while one runs answers `202` with the running rebuild.
- `GET /index/status` and `GET /control/status` report the generation, the books per state, the batches published, the size of the index, both queues and the last reconciliation.

Repeating any `POST` duplicates nothing, whether in sequence or concurrently. The state lives in MongoDB, and each transition is a conditional write that only one caller can win.

3.3 Internal contracts

In the default mode the services make no HTTP calls to each other: Hazelcast replaces the internal calls of an orchestrated design. The broker contract is documented like an endpoint (Table 7). Both messages are JSON with the same `ObjectMapper` as the API, so the broker never needs the Java classes.

```
BookTask        {"book_id": 1342, "enqueued_at": "2026-10-21T10:05:00Z"}
BatchPublished  {"batch_id": "6716f0c2a1b2c3d4e5f60718", "generation": 1,
                 "book_count": 100, "published_at": "2026-10-21T10:06:00Z"}
```

The acknowledgement and retry rules follow from one principle: a message is a hint, and MongoDB decides. A consumer acknowledges a message by claiming the book with a conditional write. If the claim does not match, the message was a duplicate or stale, and it is dropped. No ordering is assumed. An `IQueue` has no acknowledgement, so a message taken by a consumer

Table 7: Non-HTTP contract of mode `hazelcast` (`SPEC.md` §4.2). Mode `activemq` uses the same names as JMS destinations.

Destination	Message	Produced by	Consumed by
`books-to-download`, queue	`BookTask`	`control` (seeding, lost messages, expired downloads), `POST /ingest`, `crawler` retries	`crawler` workers, one book each
`books-to-index`, queue	`BookTask`	`crawler` after a download, `POST /index/update`, `control` (lost messages, released books, reindexing)	`indexer`, a batch at a time with `drainTo`
`published-batches`, topic	`BatchPublished`	`indexer`, after publishing a batch	Every `query` replica

that dies before its claim is lost. `control` detects that case when the queue is empty while a book has waited longer than `TARANTINO_REQUEUE_SECONDS`, and enqueues the book again. A lost `BatchPublished` only delays visibility, because each `query` replica also re-reads the published batches every 5 s.

In mode `orchestrator`, `control` dispatches each book with `POST /internal/download` or `POST /internal/index`, through nginx's internal port 8082. The call only offers the task to a bounded local queue of the replica. It answers `202` if the task went in, `400` for an invalid body, and `503` with `Retry-After: 1` when the queue is full, and nginx then hands the call to the next replica. `control` stops dispatching a stage at the first `503` and dispatches again any book that has not been claimed after `TARANTINO_REQUEUE_SECONDS`. It announces each new batch to every `query` replica with `POST /internal/snapshot/refresh`.

3.4 Errors

Every error is Problem Details (RFC 9457 [3]) with `application/problem+json`, produced by one handler in `common`. nginx formats its own errors the same way, including the `503` it returns when no replica answers. Table 8 lists the codes. Every `503` carries `Retry-After: 5`, and an unexpected exception gives `500` in the same format.

Table 8: Error codes of the API.

`code`	HTTP	When
`MISSING_PARAMETER`	400	`q` is missing or blank
`INVALID_PARAMETER`	400	A malformed or out-of-range parameter or `book_id`, named in `parameter`
`BOOK_NOT_FOUND`	404	The book has no state or, in `query`, is not in the snapshot
`NOT_DOWNLOADED`	409	`POST /index/update` for a book that is not downloaded
`DEPENDENCY_UNAVAILABLE`	503	MongoDB or the broker do not answer, `query` has no snapshot yet, or nginx finds no replica

3.5 Validation evidence

The `checks` module runs against the live stack and validates every response, status, header and body against the schemas of `api/openapi.yaml`. It also compares answers with `checks/expected.json`, generated from the 20 sample books. It covers everything the assignment asks for:

- 14 search cases: one word (13 matches), two words, author, language (`en`, `de`, `fr` and an unknown code), year, all filters combined, two pages of size one, an offset past the end, stopwords only and a query with no matches;
- parameter validation in the order of Table 6, and Problem Details for unknown paths and methods;
- book details, filter values, every operational endpoint, `/health` on each service port, and the idempotency of repeated `POST`s;
- in mode `orchestrator`, the internal routes, called from inside the `control` container.

It runs with one command after starting the stack, `java -jar checks/target/checks-1.0-SNAPSHOT.jar`. Continuous integration runs it in mode `hazelcast` with one `query` replica and again with three, and once in each other mode. Replicas and modes must give the same answers. The README lists the commands and the expected results.

4 Microservice Implementation

The four services share one Maven build, one hexagonal layout and one module of contracts. Every state change is a single conditional write to MongoDB, so a crash at any point leaves work that `control` can redo. This section covers the shared parts, the indexer in depth as an example, and the other services in outline.

4.1 Code structure and technologies

The system is one multi-module Maven build in Java 25. It contains `common`, the four services, and two modules that are never deployed: `bench`, with the load generator and result processing, and `checks`, with the API checks. Every dependency and plugin version lives only in the parent `pom.xml` (Table 9). Each service builds one executable jar and runs on its own, outside Compose too.

Table 9: Technologies and versions.

Piece	Choice
HTTP	Javalin 6.7, one port per service (7001 to 7004)
JSON	Jackson 2.22 with one shared `ObjectMapper`: snake_case, ISO-8601 dates, unknown fields ignored. It is Javalin's default JSON mapper, which is why it replaces the Gson of the assignment's example
Storage	MongoDB 7.0 with the synchronous Java driver 5.13; the datalake on a Docker volume
Coordination	Hazelcast 5.7, one member and every service a client; ActiveMQ Classic 6.1.8 in mode `activemq`
Packaging	One shaded jar per service and one `Dockerfile` on `eclipse-temurin:25-jre`; Docker Compose and nginx
Tests and benchmarks	JUnit 5.11, Testcontainers 2.0, JMH 1.37, XChart 3.8 for the benchmark charts

Each service keeps the hexagonal layout of Stage 1. `model` holds records and pure logic, `ports` the interfaces that logic needs, `adapters` their MongoDB, broker, datalake and HTTP implementations, and `commands` the use cases. `model` and `commands` import no Javalin, MongoDB or broker classes, so they are tested with in-memory fakes. Each service's `Main` and factory wire the pieces through constructors, without an injection framework. Every service opens its HTTP port first, so `/health` answers `503` while it waits, and then retries MongoDB and the broker without limit. Startup order therefore does not matter.

4.2 Shared contracts in `common`

Everything that two or more modules use has its literal name in `SPEC.md` and, in code, lives only in `common` (Table 10). Two services therefore cannot disagree on a collection, a field, a state or a message. `common` changes only in a pull request that all four members review. A write used by a single service, such as claiming a book or publishing a batch, stays in that service's MongoDB adapter.

Table 10: What `common` holds.

Area	Content
Model	Book, batch and rebuild states, error and problem codes, and the two messages, `BookTask` and `BatchPublished`
Configuration	`TarantinoConfig` reads every `TARANTINO_*` variable with its default and checks its invariants, such as heartbeat $\leq$ lease / 3; an invalid value stops the service naming the variable
Terms	The Stage 1 `Tokenizer` and stopwords, used by the indexer to count terms and by `query` to read `q` and `author`
Datalake	`DatalakePaths`, with the `batch` layout and the write to `<final>.<uuid>.tmp` renamed with `ATOMIC_MOVE`
MongoDB	Collection and field names, the indexes, the writes "Create the pointer", "Create the state" and "Complete a publication", and the filter of the books that block a generation switch
Messaging	The `Broker`, `BookTasks` and `BatchAnnouncements` interfaces, with their Hazelcast and ActiveMQ implementations and clients that connect in the background and keep retrying
HTTP	Problem Details for every error, `/health`, and `BookIds.parse` for every `book_id` route
Operations	`TransitionLog`, one line per state change, from which the benchmarks measure; `CrashPoints` for the recovery tests; and `LeaseHeartbeat`

4.3 Conditional writes and leases

Every state change is one conditional write to a single MongoDB document. Its filter carries the state it expects and, while a job holds the work, its `claim_id`. No service reads a document, decides in Java and writes it back. Two replicas racing for the same book or batch can therefore never both win, and no transaction is needed. `SPEC.md` §5.1 lists all 23 writes. This is the one that publishes a batch:

```
batches_g<w>.findOneAndUpdate(
  filter: {_id: B, status: "WRITING", claim_id: C,
           $expr: {$gt: ["$lease_until", "$$NOW"]}},
  update: [{$set: {status: "PUBLISHED", published_at: "$$NOW", book_ids: books}},
           {$unset: "lease_until"}],
  returnDocument: AFTER)
```

- **Server time.** Leases and expirations are compared with MongoDB's `$$NOW` inside update pipelines [5], never with a process clock, so clock drift between containers or hosts cannot break a lease.
- **One write, the right result.** When the next step depends on how the document ended up, the write is a `findOneAndUpdate` that returns it, never a write followed by a read. The published batch it returns becomes the `BatchPublished` message.
- **The heartbeat.** A job that holds a lease starts a `LeaseHeartbeat`, which renews it every `TARANTINO_HEARTBEAT_SECONDS` (20 s for a 60 s lease) on its own virtual thread. Only a renewal that does not match loses the lease; one that throws, as when MongoDB does not

answer for a moment, is retried on the next beat. Before each of its writes the job checks whether the lease is lost, and once it is lost it writes nothing else.

- **Crashes.** If a process dies or MongoDB fails halfway, nothing is marked: the lease expires and `control` redoes the work. A job never retries a write that did not match; it leaves the work to whoever won.

4.4 The indexer

The indexer turns downloaded books into metadata and postings and makes them searchable one batch at a time. Nothing a batch writes is visible until a single write publishes it. Figure 3 shows one batch.

Gathering a batch. `BatchCollector` waits for a first message on `books-to-index` and then drains the queue until it has `TARANTINO_INDEX_BATCH` books (100) or `TARANTINO_BATCH_TIMEOUT_SECONDS` (30 s) have passed since that first message. A book repeated in several messages is kept once, with its earliest `enqueued_at`, so the benchmarks can measure its queue wait.

Writing it. The indexer reads the write generation, which is the target of a running rebuild or else the current one, and ensures that generation's indexes before its first write to it. It opens the batch as `WRITING` with a new `claim_id` and a lease, starts its heartbeat, and claims the books with one `updateMany` from `DOWNLOADED` to `INDEXING`. Only the books that this write moved are indexed; another replica may have claimed the rest. `HeaderParser` reads each header and the `Tokenizer` counts the terms of each body. A book whose body is missing goes to `FAILED` with `NOT_IN_DATALAKE`. The indexer then writes `metadata_g<w>` with one `bulkWrite` of upserts and `postings_g<w>` in `bulkWrite`s of 1,000 terms, publishes, marks the books `PUBLISHED` and announces the batch.

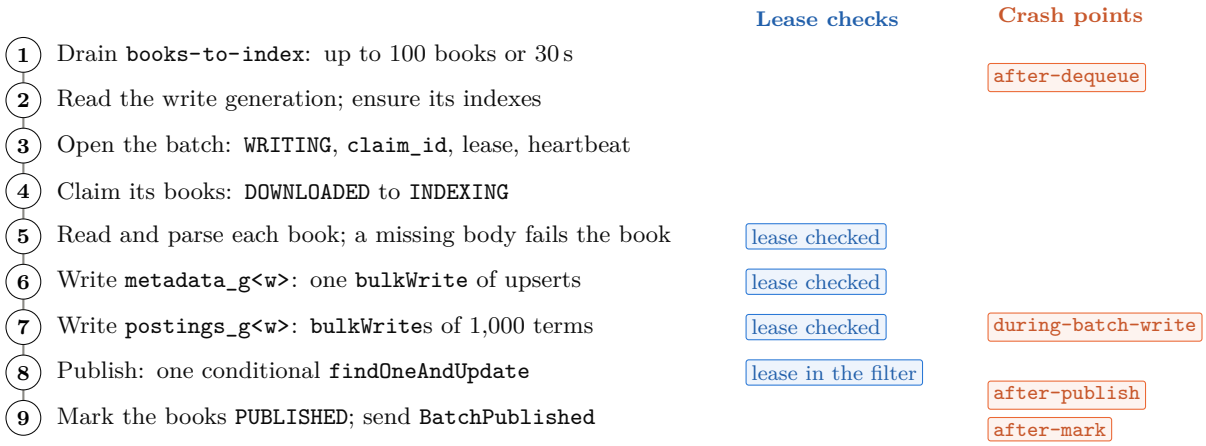


Figure 3: One batch in the indexer: nine steps, the lease checks (blue), made before each write of steps 5 to 7 and inside the publication filter, and the four crash points of the recovery tests (red), placed where each one stops the service. `during-batch-write` stops after the first postings `bulkWrite`, so the batch is really half-written, and `after-mark` stops before `BatchPublished` is sent. If the lease is lost at any check, the batch stops and is never published; if no book is left after step 5, the batch is left to expire. In both cases `control` abandons it and sends its books back to the queue.

One posting per term and batch. Stage 1 kept one document per word and added books with `$addToSet`. Here each batch writes its own document per term, with the batch's books in ascending order and each term frequency at the same position. Three properties follow. A half-written batch is invisible, because `query` only reads postings whose `batch_id` is in its snapshot of published batches. Replicas never write the same document, so frequent words cause no contention and no document grows towards MongoDB's 16 MB limit. And upserts make a repeated write harmless. The cost moves to reading: a term has one document per batch that contains it, so the batch size trades indexing throughput against the number of documents a search reads.

Metadata. `HeaderParser` keeps the title and author of Stage 1 and adds the fields of the search filters. `year` is the year of the `Release date:` line, never the eBook number on the same line, which can also have four digits. `original_year` comes from `Original publication:` when the header has it, which only 24 of the 2,800 corpus headers do. `languages` holds ISO 639-1 codes in header order, and `author_terms` the author's words through the same `Tokenizer`.

Rebuild. Each generation of the index lives in its own `metadata_g<n>`, `postings_g<n>` and `batches_g<n>`, and `index_state.current` says which one `query` serves. During a rebuild the indexer writes into the target generation while `query` keeps serving the old one. `control` sends every book back to be indexed and switches the pointer with one write once no book blocks the switch. Renaming three collections would not be atomic; switching a pointer is.

Known limitation. `POST /index/rebuild` reads the current generation and then opens the rebuild, while `control` switches the generation and then finishes the rebuild: two writes on each side. A request that falls between `control`'s two writes answers `202` but builds no new generation, so the operator sees `rebuild: null` in `GET /index/status` and sends it again. Closing the gap needs "Open a rebuild" to compute its target inside its own write, a contract change left for Stage 3.

4.5 Crawler, query and control

The other three services follow the same pattern: a claim, a lease renewed by `LeaseHeartbeat`, conditional writes, and a message only after the write.

Crawler. Each replica runs eight download workers on platform threads, and each worker takes one book at a time from `books-to-download`. A worker claims the book (`QUEUED` to `DOWNLOADING`, with its `claim_id`, a lease and one more attempt) and skips the download if the book's body is already in the datalake. Otherwise it reads the text from the local mirror or over HTTP, splits it into header and body at the Gutenberg markers and writes each file to a unique `.tmp`, renamed with `ATOMIC_MOVE`, header first. "Finish a download" then marks it `DOWNLOADED`, and only after that write is the book enqueued in `books-to-index`. A failed download goes back to `QUEUED` while attempts are left if the error may pass (`NETWORK_ERROR`, `STORAGE_ERROR`), and straight to `FAILED` if it cannot (`NOT_FOUND`, `MISSING_MARKERS`). When Gutenberg's mirror answers `429` or `503`, every download of the replica waits what `Retry-After` asks, or 1, 2, 4, 8 and 16 s.

Query. `query` keeps an immutable snapshot of the published batches of the current generation behind an `AtomicReference`. One thread refreshes it at startup, on each `BatchPublished` and every 5 s, comparing batches by `_id`, never by publication time. A search reads the snapshot once and tokenizes `q`. With filters, it first asks `metadata_g<n>` for the books matching `author_terms`, `languages` and `year`. It then reads only the postings of the snapshot's batches, intersects the terms starting with the rarest, scores each book with the sum of $(1 + \ln tf) \cdot \ln(N/df)$, breaks ties by `book_id` and reads the metadata of the page in one query. Every request therefore sees one consistent published state, and no lock is taken on the read path.

Control. At startup `control` creates the state of every candidate and enqueues only the books it created, so a restart never seeds twice. Then it reconciles every `TARANTINO_RECONCILE_SECONDS`, in six idempotent steps: (1) release expired downloads; (2) abandon the `WRITING` batches whose lease expired, in every generation; (3) settle each `INDEXING` book by its batch, releasing it if the batch is abandoned and completing its publication if the batch is published; (4) requeue the books whose message was lost, when their queue is empty, which also refills the queues after a broker restart; (5) delete the postings of batches abandoned more than `TARANTINO_GC_SECONDS` ago, on every pass, so a zombie indexer's late writes are deleted too; and (6) drive a rebuild and drop old generations. A separate task deletes orphan `.tmp` files, and `GET /control/status` reports the generation, the books per state, both queues and the last reconciliation.

4.6 Coordination modes in code

The broker sits behind three ports in `common`: `Broker` (connection and health), `BookTasks` (a work queue) and `BatchAnnouncements` (the topic). Each service's factory picks the adapters from

TARANTINO_COORDINATION, so the commands do not know which mode runs. Each mode adds little code of its own (non-blank lines of main code):

- hazelcast: 192 lines in 7 classes, IQueue and ITopic adapters and a reconnecting client.
- activemq: 328 lines in 3 classes, one JMS session and producer per queue handle, with prefetch 1 on books-to-download. With persistence, delivery is persistent and the acknowledgement is called only when the job returns, so a message held by a dead consumer goes to another one.
- polling: 104 lines in 2 worker classes plus two sorted claims, "Claim the next book" and "Claim the next book for a batch", which are findOneAndUpdates ordered by enqueued_at.
- orchestrator: 506 lines in 15 classes. control runs a tick every 200 ms that dispatches the oldest waiting books within a window of 512 downloads and 400 indexings in flight, one POST at a time, and each worker replica keeps a bounded local queue behind its internal route.

4.7 Deployment and startup

The whole system runs on one machine with Docker Compose: mongo:7.0, one Hazelcast 5.7 member in cluster tarantino, the four services built from one Dockerfile, and nginx. No service sets container_name, so docker compose up --scale crawler=3 --scale indexer=3 --scale query=2 works, while control always runs once. Only nginx publishes ports. It resolves the service names through Docker's DNS (127.0.0.11, valid=5s) with the upstream in a variable, so replicas added by --scale receive requests without restarting it, and it retries another replica when one does not answer. Every service has a healthcheck on /health and never restarts on its own, so the recovery tests decide when a crashed service comes back. One shared block gives every service the same timing variables and -Xmx512m -XX:+ExitOnOutOfMemoryError.

Two lessons from building it shaped the configuration:

- **A timeout that is not infinite.** The Hazelcast client first used the "infinite" cluster connect timeout, −1, which Hazelcast stores as Long.MAX_VALUE and subtracts the elapsed time from. On a freshly started VM the clock stepped back a few milliseconds between two attempts, the subtraction overflowed into a negative wait, and the client shut down. control then died at startup whenever Hazelcast was not ready yet. The timeout is now ten years: in practice it never gives up, and it cannot overflow.
- **A thread that dies quietly.** A batch keeps the term frequencies and postings of all its books in memory, about 0.7 MB per book. In a pilot with batches of 500 books and a 512 MB heap, the indexing thread threw OutOfMemoryError and died, but the JVM and its HTTP server did not. /health kept answering UP while the batch's books stayed INDEXING. Since then every JVM runs with -XX:+ExitOnOutOfMemoryError, so it exits and recovers like a crash, and batches stop at 250 books.

4.8 Testing and continuous integration

Tests follow the hexagonal layout: commands run on in-memory fakes, adapters on a real MongoDB in Testcontainers, and every conditional write is raced by two claimants at once. TwoAtOnce releases the same call from two threads through a barrier. For example, two indexers claiming the same books never share one, and of two publications of a batch only one matches. Publishing and abandoning at the lease boundary never both match, and two rebuild requests open one rebuild without an error. Commands are tested with a fake lease that is lost at a chosen step, and the tests check that the crash points are reached in order. Routes run on a real Javalin server with fake ports, checking status codes, Location headers, Problem Details and the exact snake_case JSON.

On 7 November 2026, mvn clean verify on dev (37576c9) ran 625 tests, all passing, in 81 s (Table 11). GitHub Actions requires five checks before anything merges into dev or main: the

build with every test, failing if the MongoDB tests were skipped; the end-to-end API checks in every mode; the shell and workflow linters; CodeQL; and the whole failure matrix with 1, 2 and 3 replicas on the benchmark corpus, which runs on every pull request to `main` and on those to `dev` that change the recovery scripts.

Table 11: Size of each module and its tests on `dev` at `37576c9`. Lines include blank lines.

Module	Main files	Main lines	Test lines	Tests
`common`	60	2,389	1,932	126
`crawler`	31	1,315	2,227	91
`indexer`	37	1,516	1,756	72
`query`	20	1,049	1,046	47
`control`	63	2,396	2,807	95
`bench`	86	5,812	3,716	180
`checks`	26	1,883	315	14
Total	323	16,360	13,799	625

5 Design Decisions

5.1 Coordination mechanism

We chose choreography over a broker for the normal path, MongoDB as the only arbiter of who owns each job, and a periodic reconciliation as the safety net (Section 2.4). Three reasons drove the choice:

- **No process in the data path.** A free worker takes a book as soon as it is queued, and replicas scale by competing for the same queue. No central process calls every service for every book, so none can become the bottleneck or the single point of failure of the pipeline.
- **Exactly one winner without transactions.** Messages can be lost or repeated, so they cannot decide anything. A conditional single-document write gives exactly one winner per transition, without locks or transactions, and it is valid across machines because it compares against MongoDB's clock.
- **Recovery from persistent state.** Every repair works from `book_states`, so a lost message, an expired lease or a lost announcement is found and fixed by reading MongoDB. Hazelcast can restart with empty queues.

The design has costs, and we accepted them knowingly. It has more moving parts than an orchestrator: leases, the heartbeat and a six-step reconciliation. It runs two systems, Hazelcast and MongoDB, where polling MongoDB alone would work. Batching adds wait, because a book waits for its batch to fill or for 30 s. And the safety net is slow on purpose: `TARANTINO_REQUEUE_SECONDS` is 120 s, and it must stay above the batch timeout so that `control` never requeues books an indexer is still holding.

5.2 The five modes measured, and the winner

The assignment allows any coordination mechanism, so we also built the four alternatives of Table 4 and measured all five in one campaign (Section 6.8). The decision rule was written on 5 November, before the campaign ran:

1. Correctness is a gate: every applicable row of the failure matrix must recover with the same answers.

2. A difference only counts when `pairs.csv` marks it as significant, that is, when the 95 % confidence interval of the difference between two modes excludes zero.
3. The criteria are then applied in this order: pipeline performance (throughput and time to publication), visibility, cost (the CPU of the coordinator and of MongoDB, the containers and the code of the mode), robustness (lost work, a broker restart, `control` down), and fit for Stage 3.
4. The default stays `hazelcast` in this delivery, and a mode the data favours is proposed for Stage 3.

Table 12 applies the rule. Every mode passes the gate: all 165 recovery runs recovered, and `recovery-compare` found the same books and answers for each row in every mode.

Table 12: Decision matrix of the five coordination modes, in the order of the decision rule. Pipeline figures are the means over 5 repetitions of `20261106-012054-coordination-variants-c1376c7`; recovery figures come from the 165 recovery runs and `20261106-073911-recovery-delay-c1376c7`. "N" is the number of `crawler` and `indexer` replicas together, and "*" marks a significant difference against `hazelcast`.

Criterion	`hazelcast`	`activemq`	`activemq-persistent`	`polling`	`orchestrator`
Gate: rows recovered	14 of 14	14 of 14	14 of 14	11 of 11	13 of 13
Throughput, books/s, N = 1 / 2 / 3	28.5 / 46.7 / 58.6	28.4 / 45.5 / 57.5	28.1 / 43.3 / 55.0*	27.5 / 47.9 / 55.5	27.7 / 44.7 / 34.6*
Time to publication p50, s, N = 1 / 2 / 3	50.0 / 30.6 / 24.6	50.9 / 32.5 / 25.7	49.1 / 31.6 / 23.1*	53.2* / 30.8 / 26.8*	54.7* / 35.3* / 30.1*
Visibility p50, ms	38–45	39–48	38–48	2,279–2,712*	168–211*
Coordinator CPU, % of a core, N = 1 / 3	3.6 / 8.3	5.4* / 12.8*	7.8* / 16.6*	3.6 / 7.0*	10.8* / 12.7*
MongoDB CPU, N = 3	224 %	219 %*	219 %	247 %*	140 %*, doing less work
Coordination containers	1, Hazelcast	1, ActiveMQ	1, ActiveMQ with a disk store	None	None
Main code of the mode	192 lines	328 lines	328 lines	104 lines	506 lines
Lost work (rows 04, 05)	`control` requeues after 11.7–11.8 s	`control` requeues after 11.7–11.8 s	Broker redelivers; reclaimed in 3.8–5.0 s	Nothing to lose	`control` redispatches after 9.7 s
Broker restart (row 12)	Queues lost; requeued after 12 s with N = 1, 82–97 s with N = 2–3	Queues lost; requeued after 11.7 s	Messages survive; reclaimed in 6.1 s	No broker	No broker
`control` down (row 11)	Pipeline goes on	Pipeline goes on	Pipeline goes on	Pipeline goes on	Pipeline stops
Stage 3 fit (argued, not measured)	Native cluster with partition backups, in memory	Network of brokers or a shared store	A shared or replicated disk store; capped at 1,460 messages/s	MongoDB load grows with every worker	One dispatcher: a bottleneck and a single point of failure
Verdict	**Winner**	Ties on performance, costs more	**Runner-up**: best recovery	Out on visibility	Out on performance

Performance. `orchestrator` loses: 41 % less throughput with 3 replicas, because one thread dispatches every book one `POST` at a time, and 4.7–5.5 s more time to publication at every N. `polling` keeps the throughput but takes 2.2–3.2 s longer to publish with 1 and 3 replicas. `hazelcast` and `activemq` never differ significantly. `hazelcast` and `activemq-persistent` only differ with 3 replicas, in opposite directions: `hazelcast` publishes 6 % more books per second, and `activemq-persistent` has a 1.6 s (6 %) lower median time to publication. Its median is lower

because persistent sends feed the crawlers more slowly than they take books. A book waits 1 ms in `books-to-download` instead of 0.8 s, and slightly less in `books-to-index`, so part of the gain is a later start of each book's clock, not faster work. The same throttling costs the 6 % of throughput. Performance therefore leaves three modes in a tie.

Visibility and cost. `hazelcast`, `activemq` and `activemq-persistent` make a batch searchable within 38–48 ms, so visibility does not separate them. Cost does: `hazelcast`'s coordinator uses significantly less CPU than both ActiveMQ modes at every N, 35 % less than `activemq` and 50 % less than `activemq-persistent` with 3 replicas. It also needs no disk store and has the least code of the three. The rule stops here: **`hazelcast` wins**, and it remains the mode of Stage 2.

What the rule leaves to Stage 3. Robustness, the criterion after cost, favours `activemq-persistent`. It is the only mode whose lost work comes back without `control`: the broker redelivers what a dead consumer did not acknowledge, and the book is claimed again 3.8–6.1 s after the failure, against 11.7 s for a requeue by `control`. If robustness came before cost, `activemq-persistent` would win. That is why its property, not its broker, goes into the Stage 3 plan (Section 7.2). `control` stays necessary in every mode, because it also releases expired leases and drives rebuilds, and persistence caps the broker at about 1,460 messages/s. At the 500–670 books/s that ingestion reaches, with two messages per book, that cap is close.

5.3 Single-machine deployment

Everything runs on one host with Docker Compose, as the assignment requires. Addresses are Compose service names, the datalake is a local volume, and there is one MongoDB server and one broker. Ports, addresses, data paths, replica counts, thread counts and every timing are environment variables with working defaults, validated at startup. Replicas are added with `--scale` and need no restart, because nginx resolves service names on every request through Docker's DNS. Only nginx publishes ports, and the front-end and operational ports are separate, so a front-end can never reach an administrative route.

5.4 API design

- **One search endpoint** with optional filters and pagination, as the assignment asks, plus book details and filter values for the front-end use cases.
- **Asynchronous state changes**: `202` with a `Location` to poll, or `200` if already done. Handlers stay in the millisecond range, cannot exhaust the HTTP pool, and repeated or concurrent requests duplicate nothing.
- **Consistent reads**: every search reads one immutable snapshot of published batches, so a request never mixes two states of the index, and a rebuild switches the whole snapshot at once.
- **Problem Details for every error**, including nginx's, with a stable `code` and the failing `parameter`, so a client handles errors without parsing text.

5.5 Storage

The datalake keeps the Stage 1 `batch` layout, which won the Stage 1 comparison: $O(1)$ store and lookup with one directory per 1,000 ids. The inverted index stays in MongoDB, also chosen in Stage 1, but with one postings document per term and batch instead of one per term (Section 4.4). Metadata moves from SQLite to MongoDB: SQLite allows one writer at a time, which several indexer replicas cannot share, and MongoDB avoids running a third database for it. Book states also live in MongoDB, because conditional single-document writes are exactly what claims and leases need. Each generation of the index has its own collections, so a rebuild never disturbs what `query` serves.

5.6 Concurrency model and configuration

Replicas coordinate only through conditional writes and the broker, and threads inside a process share only immutable snapshots and `java.util.concurrent` structures, so no lock is ever taken. Table 13 gives the configuration we keep and the measurement behind each setting.

Table 13: Configuration decisions and the evidence behind them (Section 6).

Setting	Kept	Evidence
Books per batch	100; 250 with a larger heap	250 gives 17 % more books/s than 100, but the indexer container peaks at 745–808 MiB with `-Xmx512m`; 500 ran out of memory
`indexer` replicas	As many as MongoDB allows	Indexing limits the pipeline; 2 and 3 replicas give ×1.70 and ×2.10, while MongoDB's CPU grows faster than the indexers'
`crawler` replicas and workers	1 replica, 8 to 16 workers	Replicas add nothing (521–548 against 504 books/s); 16 workers add 24 % over 8; one crawler already downloads about 18 times faster than one indexer publishes
`query` replicas	2	The second replica adds 29 % requests/s and cuts latency by 22 %; the third adds nothing
HTTP threads	Javalin's default, 8 to 250	128 threads add nothing over 32 and lose 4 % of the requests/s; 8 threads with 32 clients raise latency by 77 %
Batch announcements	Topic listener plus a 5 s refresh	A batch is visible in 44 ms on average with the listener and 2.6 s without it, at no measurable CPU cost

5.7 Smaller decisions

Table 14 lists the smaller decisions that shaped the code, each with the alternative it rejected.

Table 14: Smaller decisions, with the alternative each one rejected.

Decision	Why	Rejected
Java 25	Allowed by the course; virtual threads carry the lease heartbeats cheaply, and records model messages and snapshots	Java 17 of the example
Jackson	Javalin's default mapper; one `ObjectMapper` for the API, MongoDB documents and broker messages	Gson of the example
Generation pointer for rebuilds	Switching it is one atomic write	Renaming three collections, which is not atomic
One conditional write per transition, on `$$NOW`	Single-document atomicity needs no transactions, and MongoDB's clock is the only clock	Read, decide in Java, write
A lease is lost only by a renewal that does not match	A moment without MongoDB should not cost a batch	Losing the lease on any exception
Every JVM exits on `OutOfMemoryError`	A dead thread behind a healthy `/health` is never recovered	Catching `Error` in the worker
Ten-year Hazelcast connect timeout	The infinite one overflows when the clock steps back	−1
JMH with three forks and the protocol in annotations	The error covers differences between JVMs, and every way of launching keeps the protocol	One fork, protocol in a runner

6 Benchmarks and Results

6.1 Setup and methodology

Every figure in this section comes from one official campaign on one machine (Table 15). The script `scripts/bench/all.sh` ran every experiment and the whole failure matrix in the five modes, in 187 steps and 9 h 45 min, from a clean commit of `dev`. Only one step failed: `broker-latency`, because Git Bash rewrote the folder it passed to the container. Once the script passed a relative path, that step ran again alone, from a commit that only changed scripts. Appendix A gives the run, command and raw data behind every table and figure.

Table 15: Host and fixed conditions of every official run, as each `manifest.json` and `corpus.json` record them.

Host	Intel Core Ultra 9 275HX, 24 hardware threads (8 P-cores and 16 E-cores), 31.4 GiB, 1 TB disk, Windows 11
Containers	Docker Desktop 29.9.2 with 24 CPUs and 15.3 GiB; every service JVM with `-Xmx512m`
Corpus	2,800 books and 755,974 distinct terms from the local mirror, checked against `workload/corpus.sha256` before every run, so the network is never measured
Campaign	`20261105-215348-campaign-c1376c7`, from `dev` at `c1376c7`, overnight on 5–6 November 2026

- **One factor at a time.** Each experiment varies one factor around a baseline, and its script prints the hypotheses to the run's `log.txt` before measuring.
- **Five interleaved repetitions.** Repetition 1 of every configuration runs first, then repetition 2, and so on, so any drift over the night, such as heat, spreads over every configuration instead of falling on one.
- **Statistics.** Tables give the mean ± the half-width of its 95 % confidence interval over the repetitions, with Student's t for $n-1$ degrees of freedom. Percentiles pool the values of every repetition: books, `docker stats` samples or requests.
- **Three levels of concurrency, kept apart.** Client concurrency is the closed-loop clients of the load generator; threads are the workers or HTTP threads inside one process; replicas are processes. Each table states the three, and each trial records them in `manifest.json`.

To fit the campaign into one night, it reused two runs from 3 and 4 November on the same machine, because `git diff` shows that the code they measure did not change. These are `search-latency` and the indexer's JMH. The coordination cost comes from the `hazelcast` trials of `coordination-variants`, which follow the same procedure. Rows 09 to 12 of the failure matrix ran with one replica only outside `hazelcast`, because their time is mostly processing the whole corpus, which `coordination-variants` already measures for every mode and replica count.

6.2 Microbenchmarks

JMH [8] runs in each service's `src/test/java` with 5 warmup and 10 measurement iterations, three forks and a fixed heap, so its error also covers differences between JVMs. Each bound in Table 16 was stated before measuring and is checked with at least three sizes.

The tokenizer costs 10.4–11.3 ms per MB across a 168-fold range of sizes, so it is linear, and the header parser takes about 7 µs whatever the book. Writing a batch costs less per book from 10 to 1,000 books, because books share terms and each term gets one postings document per batch. The single book is cheaper only because it is book 1, an 11 KB body. Splitting a book in the crawler takes at most 12 ms, and ranking takes 60 µs. None of these operations is what limits the pipeline, as the next sections show.

Table 16: JMH results, average time per operation ± JMH's 99.9 % error. Indexer: `20261103-204454-index-throughput-6925d4e`; crawler: `20261105-215351-ingest-throughput-c1376c7`; query: `20261104-140900-search-latency-8e1c1e8`.

Benchmark	Input	Time	Per unit	Bound
`Tokenizer`, terms of a body	Book 1080, 20 KB body	0.206 ± 0.004 ms	10.4 ms/MB	Linear in the body: holds
	Book 84, 429 KB	4.83 ± 0.01 ms	11.3 ms/MB	
	Book 2600, 3.34 MB	36.7 ± 0.1 ms	11.0 ms/MB	
`HeaderParser`	Book 1080	6.9 ± 0.9 µs	–	Constant: holds
	Book 84	7.54 ± 0.09 µs	–	
	Book 2600	7.09 ± 0.03 µs	–	
Writing a batch to MongoDB	1 book	19.8 ± 0.7 ms	19.8 ms/book	Cost per book falls with the batch: holds
	10 books	338 ± 3 ms	33.8 ms/book	
	100 books	2,806 ± 33 ms	28.1 ms/book	
	1,000 books	12,535 ± 175 ms	12.5 ms/book	
Crawler: split header and body	Book 1080, 40 KB	102 ± 10 µs	–	Linear in the text
	Book 84, 450 KB	1.62 ± 0.01 ms	–	
	Book 2600, 3.4 MB	12.3 ± 0.4 ms	–	
Query over 1,000 books	TF-IDF ranking, 3 words	60.2 ± 0.4 µs	–	About 1 % of a request end to end
	Search without filters	44.9 ± 0.2 µs	–	
	Search with filters	12.3 ± 0.1 µs	–	

6.3 Ingestion throughput

Table 17: Books downloaded per second by `crawler` replicas and download workers per replica, over the whole window (first claim to last download) and the steady window (10 % to 90 % of the books). `20261105-215351-ingest-throughput-c1376c7`.

Replicas	Workers per replica	Whole window (books/s)	Steady 10–90 % (books/s)
1	4	370 ± 51	393 ± 51
1	8	504 ± 135	560 ± 132
1	16	623 ± 27	671 ± 44
2	8	548 ± 73	603 ± 84
3	8	521 ± 17	572 ± 17

- **Workers help with a shrinking gain** (Table 17): +36 % from 4 to 8 and +24 % from 8 to 16. The curve bends but does not flatten by 16 workers, as the hypothesis expected.
- **Replicas do not help**, against the hypothesis: 2 and 3 replicas of 8 workers download as many books per second as one, within their intervals.
- **CPU is not the limit.** During ingestion the host uses 956–1,281 % of the 2,400 % available (Table 20), and more crawlers raise their total CPU from 378 % to 537 % without more books per second. Splitting a book takes at most 12 ms, while each of 8 workers has 16 ms per book at 504 books/s. The limit is in what every crawler shares: the one `books-to-download` queue, the one MongoDB and the datalake volume. These runs do not tell which of the three.

6.4 Indexing throughput and latency

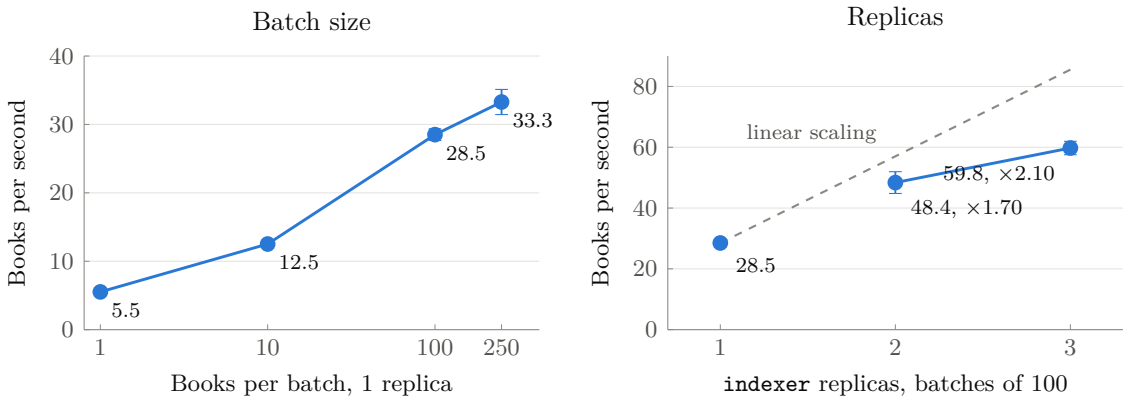


Figure 4: Books indexed per second over a whole rebuild of the 2,800 books, mean and 95 % confidence interval of 5 repetitions. Left: batch size with one replica. Right: replicas with batches of 100, against linear scaling from one replica. `20261105-220915-index-throughput-c1376c7`.

- **Bigger batches pay off, then flatten** (Figure 4), as expected: ×2.3 from 1 to 10 books, ×2.3 from 10 to 100, and only +17 % from 100 to 250. Each batch pays a fixed cost (its lease, publication and marking) that more books share.
- **Replicas help less than linearly**, as expected: ×1.70 with 2 and ×2.10 with 3, that is 85 % and 70 % of linear.
- **Batches of 250 fit in a 512 MB heap**, as expected: no indexer ran out of memory, and the container peaked at 745–808 MiB per trial with batches of 250, against 587–778 MiB with batches of 100.
- **MongoDB is where the time goes.** JMH writes a batch of 100 books in 2.81 s, which alone caps one indexer at 36 books/s, close to the 28.5 measured end to end. Counting the terms of the largest book takes 37 ms. During indexing MongoDB's CPU grows faster than the indexers' (Table 20).

Indexing latency. Over a whole rebuild, each book costs the inverse of the throughput: 0.18 s with batches of 1, 0.035 s with 100 and 0.017 s with three replicas of 100. A single book also waits for its batch: from its claim to its batch's publication it takes 3.45 s with one replica and 4.93 s with three (Table 19).

6.5 Search latency under concurrency

Table 18: `GET /search` by clients, HTTP threads and `query` replicas, cycling through `workload/queries.txt`; 18,496,487 requests in all. `20261104-140900-search-latency-8e1c1e8`.

Clients	HTTP threads	Replicas	Mean latency (ms)	p95 (ms)	Requests/s
1	32	1	1.54 ± 0.02	2.73	646 ± 8
8	32	1	2.20 ± 0.06	3.87	3,618 ± 99
32	32	1	7.29 ± 0.04	14.5	4,374 ± 26
64	32	1	14.5 ± 0.2	27.2	4,381 ± 50
32	8	1	12.9 ± 1.3	19.3	2,493 ± 254
32	128	1	7.56 ± 0.06	15.1	4,217 ± 36
32	32	2	5.66 ± 0.02	10.1	5,625 ± 24
32	32	3	5.82 ± 0.04	10.5	5,471 ± 39

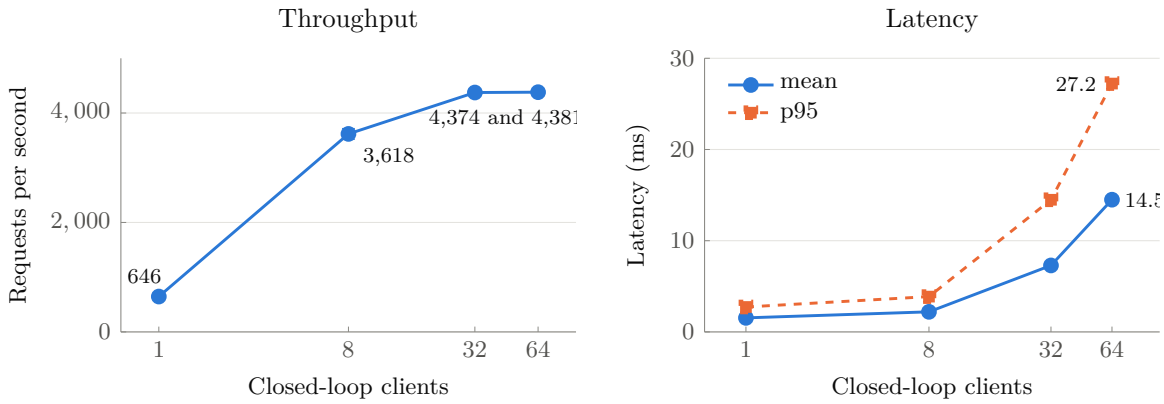


Figure 5: `GET /search` with one `query` replica and 32 HTTP threads, as the closed-loop clients grow. Mean and 95 % confidence interval of 5 repetitions of 120 s each, after 30 s of warmup; the p95 is the mean of each repetition's p95. `20261104-140900-search-latency-8e1c1e8`.

- **One replica saturates at about 4,400 requests/s** with 32 clients (Figure 5). From 32 to 64 clients the throughput stays the same and the latency doubles, which is Little's law [9] for a saturated server: 64 clients / 4,381 requests/s = 14.6 ms, the measured 14.5 ms.
- **Threads must cover the clients.** With 8 HTTP threads and 32 clients, requests queue and latency is 77 % higher than with 32 threads; 128 threads add nothing and lose 4 % of the requests/s (Table 18).
- **A second replica helps, a third does not.** Two replicas give +29 % requests/s and −22 % latency; three give the same as two. With two replicas the limit is no longer the `query` JVM but what the replicas share: nginx, MongoDB and the host's CPUs, which the load generator also uses.
- **The ranking is not the cost.** JMH ranks a three-word query in 60 µs, about 1 % of the 5–7 ms each request takes. The rest is reading postings and metadata from MongoDB, HTTP and JSON.
- **Answers stay correct.** Every request answered `200` except 170 `503`s, 168 of them with 64 clients: 0.006 % of that configuration's 2,628,660 requests.

6.6 Coordination cost

Table 19: Where a book's time goes in mode `hazelcast`, by replicas of `crawler` and `indexer` together: the wait in `books-to-download` and in `books-to-index`, the work, and the time from enqueue to publication. Mean of 5 repetitions ± 95 % interval, from the transition logs. `hazelcast` trials of `20261106-012054-coordination-variants-c1376c7`.

N	Wait for a crawler	Download	Wait for an indexer	Indexing	Enqueue to publication
1	1,383 ± 327 ms	12.6 ± 1.6 ms	46.6 ± 1.9 s	3.45 ± 0.13 s	51.4 ± 1.9 s
2	670 ± 115 ms	27.0 ± 3.8 ms	27.0 ± 2.7 s	4.19 ± 0.41 s	31.9 ± 3.1 s
3	736 ± 127 ms	44.4 ± 4.5 ms	19.9 ± 0.7 s	4.93 ± 0.21 s	25.6 ± 0.9 s

- **Coordination costs far more time than the work**, as expected (Table 19): with one replica a book waits 1.4 s for a crawler to download it in 13 ms.
- **More consumers on one queue help little.** The wait for a crawler halves with 2 replicas and does not fall further with 3, while the download itself gets slower with every replica, 13, 27 and 44 ms.

- **Indexing is the bottleneck of the pipeline.** The wait in `books-to-index` is 91 % of the time to publication with one replica. It falls 42 % with 2 replicas and 57 % with 3; from 1 to 3 replicas that is more than the 47 % of the download wait, against the hypothesis. Books wait for an indexer, not for their batch to fill: with one replica the p95 of that wait is 87 s, far above the 30 s a batch waits at most.

6.7 Resource usage

Table 20: CPU, in percent of one core as `docker stats` counts it, and memory during each phase, by replicas of `crawler` and `indexer`. "Stack" adds up every container; "Host" is the whole machine. `20261105-235758-resource-usage-c1376c7`.

N	Phase	Crawlers' CPU	Indexers' CPU	MongoDB's CPU	Stack CPU	Stack memory (MiB)	Host CPU
1	Ingestion	378 ± 94 %	–	50 ± 9 %	505 ± 105 %	1,199 ± 122	956 ± 85 %
2	Ingestion	512 ± 74 %	–	62 ± 11 %	680 ± 109 %	1,398 ± 125	1,163 ± 155 %
3	Ingestion	537 ± 53 %	–	65 ± 4 %	717 ± 32 %	1,723 ± 73	1,281 ± 71 %
1	Indexing	–	64 ± 2 %	77.9 ± 0.4 %	147 ± 2 %	2,264 ± 199	208 ± 7 %
2	Indexing	–	119 ± 10 %	150 ± 3 %	277 ± 7 %	2,967 ± 148	346 ± 12 %
3	Indexing	–	176 ± 11 %	220 ± 3 %	406 ± 10 %	3,930 ± 134	494 ± 11 %

- **Crawlers share a fixed rate.** Their total CPU grows with replicas, but each crawler uses less, 378, 256 and 179 %, instead of staying flat as expected: the same books per second are split among more crawlers, and the 24 hardware threads are never near saturation.
- **MongoDB follows the indexers.** During ingestion the crawlers use 7.6–8.3 times MongoDB's CPU; during indexing MongoDB uses more CPU than the indexers and grows with them, from 78 % to 220 %, as expected.
- **Memory is bounded.** A crawler or indexer container reaches 497–683 MiB at p99, above its `-Xmx512m` in all but one configuration, because `docker stats` counts the whole JVM (heap, metaspace, thread stacks and buffers), not the heap. No container grows with the load; `control` and `query` stay below 270 MiB.
- **The host pays for Docker.** The host uses 1.7–1.9 times the stack's CPU during ingestion and 1.2–1.4 times during indexing: the rest runs in Docker Desktop's VM and Windows.

6.8 Coordination mechanisms compared

`coordination-variants` ran the whole corpus through the five modes of Table 4, with 1, 2 and 3 replicas of `crawler` and `indexer` together, 5 interleaved repetitions each: 75 trials. Its `pairs.csv` holds the difference of every pair of modes, for every metric and replica count, with its 95 % confidence interval. Figure 6 shows the pipeline metrics, and Table 12 in Section 5.2 the rest.

The script stated one hypothesis per mode before measuring. The common hypothesis, that the work per book is the same in every mode, holds for indexing but not for downloads. In `activemq-persistent` and `orchestrator` a download takes 8–15 ms at p50 with 2 and 3 replicas, against 21–45 ms elsewhere, because books reach the crawlers more slowly than the crawlers take them, so fewer downloads overlap. That changes milliseconds against a time to publication of tens of seconds.

- `activemq` has the throughput and time to publication of `hazelcast` at every N, within their intervals, as expected, with 1.5 times its coordinator CPU.
- `activemq-persistent` has the throughput of `activemq` at every N, as expected, with 1.3–1.4 times its broker CPU.

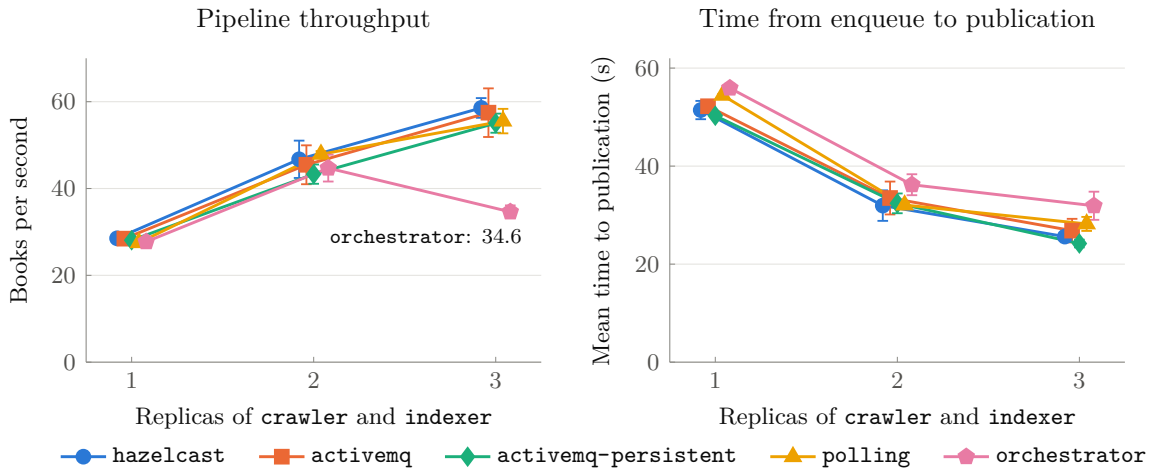


Figure 6: The same pipeline in each coordination mode: books published per second over the whole corpus, and mean time from a book's enqueue to the publication of its batch. Mean and 95 % confidence interval of 5 repetitions; the modes are shifted slightly along the x axis so their intervals do not overlap. `20261106-012054-coordination-variants-c1376c7`.

- **`polling`** has the throughput of **`hazelcast`** at every N and 10–12 % more MongoDB CPU, from the sorted claims and the idle polls. Visibility at p50 is 2.3–2.7 s, about half the 5 s refresh period. All three matched the hypotheses.
- **`orchestrator`** keeps the throughput with 1 and 2 replicas and loses 41 % with 3, which its hypothesis allowed, because one thread sends one `POST` at a time. Its time to publication is 4.3–6.3 s above **`hazelcast`**, not the two ticks expected: books wait in MongoDB until `control` dispatches them, 4.2–5.6 s against 0.7–1.4 s in `books-to-download`. Its visibility is 173–208 ms at p50, under the expected 300 ms.
- **Coordination is cheap in every mode.** The broker or `control` uses at most 17 % of one core, against MongoDB's 80–247 %.

Publication visibility. The time from a batch's publication to the moment a `query` replica holds it in its snapshot depends on how `query` learns of the batch (Table 21). With the topic listener a batch is searchable in about 40 ms, under the expected 100 ms. Without it, delays spread between 0 and the 5 s period, with a median of 2.7 s. The listener costs no CPU that the run can see: each `query` replica uses about 3 % of a core either way.

Table 21: Publication visibility with 2 `query` replicas, one value per batch and replica. `20261106-002208-publication-visibility-c1376c7`.

`published-batches` listener	Mean (ms)	p50 (ms)	p95 (ms)	p99 (ms)
On	43.6 ± 7.3	37	54	145
Off	2,605 ± 251	2,672	4,880	4,965

Broker latency in isolation. `broker-latency` sends 30,000 messages per trial through each broker, with 1, 3 and 8 consumers (Figure 7). Every p50 is between 0.1 and 0.2 ms: 102–105 µs for Hazelcast, 137–139 µs for ActiveMQ and 180–185 µs for persistent ActiveMQ. Persistence costs 1.3 times the latency, not the 3 times expected, but it caps the throughput at about 1,460 messages/s whatever the consumers, because each send waits for its disk write [7]. Non-persistent ActiveMQ scales with consumers, ×2.8 from 1 to 8, while Hazelcast stays at about 8,700 messages/s. A message is never what a book waits for: the slowest p50, 185 µs, is about 7,700 times shorter than the median wait in `books-to-download` with one replica.

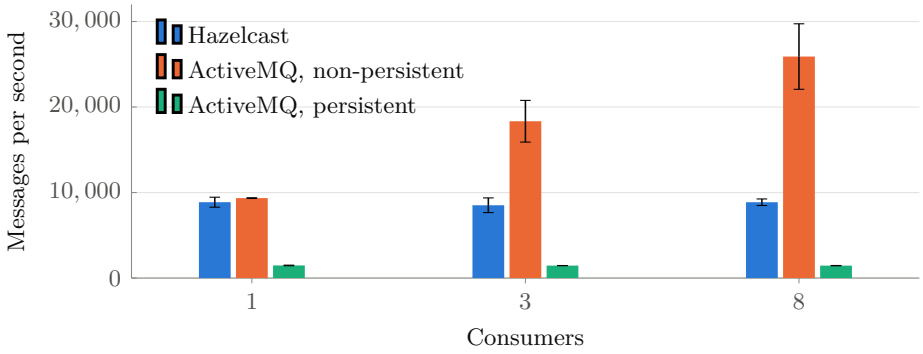


Figure 7: Throughput of each broker in isolation, mean and 95 % confidence interval of 5 repetitions of 30,000 messages: Hazelcast 8,513–8,872 messages/s, non-persistent ActiveMQ 9,350, 18,338 and 25,904, persistent ActiveMQ 1,454–1,475. `20261106-123534-broker-latency-b4667d8`.

6.9 Recovery

Each of the 165 recovery runs causes one failure of the matrix (Appendix C) and measures the time until every book is `PUBLISHED`, found exactly once by `/search` and in a single published batch (Table 22). The runs use the short leases of `scripts/recovery/recovery.env` (6 s) and run once per row, mode and replica count, so they have no confidence interval. `recovery-compare` checks that the runs of a row give the same books and search answers whatever the replicas and mode, and it passed for all 14 rows.

Table 22: Seconds from the failure to correct answers. Rows 09–12 seed the whole mirror, so their time includes processing the 2,800 books; the others seed the 20 sample books. Runs `2026110[56]-...-recovery-<nn>[-<mode>]-r<N>-c1376c7`, summarized by `20261106-073908-recovery-summary-c1376c7`.

Rows	Failure	Corpus	`hazelcast`, N = 1/2/3	Other modes
01–04	`crawler` crash points	sample	8.6–18.8	7.2–22.8
05–08	`indexer` crash points	sample	6.3–17.7	6.6–16.9
09	`crawler` paused longer than its lease	mirror	373 / 276 / 225	363–373, N = 1
10	`indexer` paused longer than its lease	mirror	382 / 278 / 241	368–441, N = 1
11	`control` killed while seeding	mirror	367 / 255 / 253	365–373, N = 1
12	Broker restarted	mirror	374 / 261 / 235	372 and 384, ActiveMQ modes, N = 1
13	MongoDB stopped for 10 s	sample	16.6 / 15.9 / 14.8	14.7–22.6
14	A `query` replica killed	sample	– / 0.9 / 0.7	0.2–1.2

Every row recovers in every mode where it applies, with the same answers. The crash points of `crawler` and `indexer` recover within 23 s with the 20 sample books. Rows 09–12 take 225–441 s, mostly ingesting and indexing the corpus, and that time falls with N as indexing gets faster. Killing one of two or three `query` replicas costs at most 1.2 s of search answers, because nginx sends the requests to the others.

How lost work comes back. `recovery-delay` reads the 29 runs of rows 04, 05 and 12, where a crash or a broker restart loses work after its message was taken, and measures for each book the time from the failure to its requeue and to its next claim (Table 23). In `hazelcast` and `activemq`, `control` requeues 10.8–11.8 s after the failure, as expected. Row 12 of `hazelcast` is the exception: with 2 and 3 replicas the median requeue takes 97 and 82 s, against 12 s with one, and these runs do not tell why. `activemq-persistent` requeues nothing: the broker redelivers, and books are claimed again in 3.8–6.1 s at the median. The orchestrator dispatches again after 9.5–10.1 s, because its 10 s count from the book's last dispatch, which comes before the crash.

Table 23: Median seconds from the failure to a book's requeue (or new dispatch) and to its next claim, over the books of every run of the row. `20261106-073911-recovery-delay-c1376c7`.

Mode	Rows 04 / 05: requeue	Rows 04 / 05: claim	Row 12: requeue	Row 12: claim
`hazelcast`	11.8 / 11.7	11.5 / 5.6	82.4	12.3
`activemq`	11.8 / 11.7	10.8 / 8.5	11.7	13.5
`activemq-persistent`	none	3.8 / 5.0	none	6.1
`orchestrator`	9.7 / 9.7	5.0 / 9.8	–	–

6.10 Bottlenecks and the limits of this evidence

1. **Indexing limits the pipeline.** One indexer publishes 28.5 books/s, while one crawler downloads about 500. Books therefore wait 20–47 s in `books-to-index`, and an indexer spends its time writing postings to the one MongoDB.
2. **Ingestion stops at 500–670 books/s** with 8 or more workers, whatever the replicas, with at most 53 % of the host's CPU in use. The shared queue, MongoDB and the datalake volume limit it.
3. **Search saturates one `query` replica at about 4,400 requests/s.** A second replica adds 29 %, and a third adds nothing.
4. **Coordination is not a bottleneck in any mode.** A message takes 0.1–0.2 ms, and the broker or `control` uses less than 17 % of a core. The mode changes how soon a book is searchable: about 40 ms with a listener, 170–210 ms with the orchestrator, and 2.3–2.7 s with polling.

These results describe one host. Every replica shares its CPUs, its disk, and the one MongoDB and broker, so a speedup here shows how the design uses one machine, not how it will scale across several. The evidence has other limits too:

- `search-latency` and the indexer's JMH come from 3 and 4 November, reused because their code did not change.
- After the campaign, `dev` changed one class of `control` (`65095a7`): the orchestrator now asks `query` to refresh once per tick with new batches, not once per batch. Only the visibility of mode `orchestrator` can differ from what was measured at `c1376c7`.
- The campaign left an idle ActiveMQ container running during the recovery runs of `polling` and `orchestrator`. No service of those modes connects to it, and the scripts bring every mode down since `b4667d8`.
- Docker Desktop on Windows adds a VM: the host uses 1.2–1.9 times the CPU of the containers. `docker stats` reports a container's whole memory, not the JVM heap.
- An ingestion phase lasts about 5 s, about 10 `docker stats` samples per repetition, so its CPU means are coarse. Recovery runs have one sample per row, mode and replica count, and in the four alternative modes rows 09–12 only ran with one replica.

7 Conclusions and Future Improvements

7.1 What the measurements say

Strengths. The design is correct under every failure we could cause. All 165 recovery runs recover with the same answers whatever the mode and the replica count, and no book ever appeared twice. Coordination costs little: a message takes 0.1–0.2 ms, and the broker uses at most 17 % of one core. Indexer and `query` replicas raise throughput on one host, ×2.10 with three indexers and +29 %

with a second `query`. A book becomes searchable about 40 ms after its batch is published. The contracts held: the API checks pass in every mode and with one or three `query` replicas, and every figure traces to a run.

Weaknesses. Indexing is the bottleneck: one indexer publishes 28.5 books/s against about 500 downloaded, and its time goes into MongoDB writes. Crawler replicas add nothing beyond 500–670 books/s, because they share one queue, one database and one volume. `control` is a single process that every mode needs, since it alone releases leases and drives rebuilds. When the broker loses messages, `control` brings them back after `TARANTINO_REQUEUE_SECONDS`, slower than the redelivery of a persistent broker. One open question is why that requeue took 82–97 s after a Hazelcast restart with 2 and 3 replicas, against 12 s with one.

7.2 From one host to the Stage 3 cluster

The conditional writes already work across machines. Claims, leases and expirations compare with MongoDB's clock, never a host's, and no process holds a lock that another needs. What Stage 3 has to replace is everything named after the host (Table 24): Compose service names and nginx's resolver `127.0.0.11`, the local `datalake` volume, the single Hazelcast member and MongoDB server, and, in mode `orchestrator`, the JVM's DNS cache through which `control` finds the `query` replicas.

Table 24: Each measured limit, the single-host assumption behind it, and the change Stage 3 makes.

Measured limit	Single-host cause	Stage 3 change
Indexing: 28.5 books/s per indexer, ×2.10 with 3, with MongoDB's CPU growing faster than the indexers'	Every indexer writes postings to one MongoDB server	Postings sharded by `term` across a sharded MongoDB, so indexers on several nodes write to different shards; the batch, its lease and its publication stay single-document writes
Ingestion: 500–670 books/s whatever the replicas, with the CPU far from full	One queue on one Hazelcast member, one MongoDB, one datalake volume	A Hazelcast cluster with the queue partitioned by `book_id`, and a datalake on shared storage, or one per node with the book's node in `book_states`
Search: one `query` replica saturates at about 4,400 requests/s, and a third adds nothing to two	Replicas share nginx, the one MongoDB and the host's CPUs with the load generator	`query` replicas on several nodes behind a load balancer, reading a MongoDB replica set; the snapshot already lets each replica answer on its own
Recovery: lost messages wait for `control`'s requeue	`control` is one process, acceptable on one host	`control` on several nodes with one leader, elected through a lease document in MongoDB; lost work acknowledged after the job (below)
Mode `orchestrator`: 41 % less throughput with 3 replicas	One thread dispatches every book, one `POST` at a time	Not carried to Stage 3

Communication and storage across machines. Services keep talking only through the broker and MongoDB, so moving them to other nodes changes addresses, not code. Hazelcast becomes a cluster of three members, where each queue item has a synchronous backup on another member, so one member can fail without losing queued work. MongoDB becomes a replica set, with majority writes for every conditional write, so a failover never loses a claim. The datalake moves to storage that every node mounts.

Distribution of tasks and requests. Crawlers and indexers keep competing for queues, which needs no coordination between them. Partitioning `books-to-download` and `books-to-index` by `book_id` removes the single queue owner that limits ingestion today. Search requests are spread by a load balancer, as nginx does now.

Consistency and unavailable nodes. Publication stays one conditional write, and `query`

keeps reading a snapshot of published batches, so a reader never sees a half-built index on any node. A node that disappears leaves leases that expire, and `control` releases them exactly as it does after a crash today: rows 01 to 10 of the failure matrix already exercise that path. A network partition cannot create two owners of a book, because ownership is a MongoDB document, not a broker message.

7.3 The winning mode in Stage 3

`hazelcast` wins under the decision rule fixed before the campaign (Section 5.2), so Stage 3 keeps it and does not migrate to another broker. The runner-up, `activemq-persistent`, showed the one property worth taking: lost work comes back without `control`, in 3.8–6.1 s instead of about 12 s, because the message is acknowledged only after the job. Stage 3 adds that property to the Hazelcast design and measures it with the same rows and the same `recovery-delay` experiment:

1. Take each book inside a Hazelcast transaction that commits after the claim and the job, so a consumer that dies returns its message to the queue. Hazelcast's transactional queue offers that rollback [6], and the claim in MongoDB still drops any duplicate it produces.
2. Explain and fix the 82–97 s requeue after a broker restart with several replicas (row 12).
3. Elect the `control` leader through a lease in MongoDB, so reconciliation survives the loss of a node.
4. Repeat `coordination-variants` for `hazelcast` and `activemq-persistent` on the cluster. If the transactional queue does not close the gap in rows 04, 05 and 12, persistent ActiveMQ replaces Hazelcast as the broker. The rest of the code stays the same, because both already sit behind the same ports.

Smaller improvements follow from this report: batches of 250 books once the indexer has a larger heap, which gave 17 % more throughput, and "Open a rebuild" computing its target inside its own write, which closes the rebuild race of Section 4.4.

7.4 Lessons

- **Measure the alternatives.** Arguing against orchestration and polling was easy; measuring them showed where each one actually loses, by how much, and what the runner-up does better.
- **Keep the truth in one place.** Treating every message as a hint and every transition as one conditional write made 14 different failures recoverable by the same reconciliation.
- **Keep the evidence with the results.** Five interleaved repetitions, hypotheses written before measuring, and a versioned run behind every figure made every number in this report checkable against its raw data, and we checked them all before submitting it.

8 Statement on AI Usage

In accordance with the course guidelines, we disclose the use of AI tools during this stage, mainly Claude (Anthropic) through Claude Code. AI was used as an assistant to speed up routine work. The architecture, the contracts in `SPEC.md`, the design of every experiment, the decision rule for the coordination modes and the interpretation of the results were decided and reviewed by the team. Specific uses included:

- **Code and tests.** Drafting and reviewing implementation code, tests and scripts against `SPEC.md`, including parts of the alternative coordination modes. Every change went through a pull request and the five required checks of continuous integration before it reached `dev`.

- **Reviews and audits.** Checking the repository against the assignment and the specification, and finding inconsistencies between the documents, the code and the results.
- **Data processing and writing.** Extracting tables from the run files, checking every figure in this report against the raw data, and drafting and formatting the documentation and this report in LaTeX.

Every piece of AI-assisted code or text was reviewed, tested and integrated by the team member who owns it, to keep it consistent with the specification and with academic integrity.

References

[1] Big Data, Universidad de Las Palmas de Gran Canaria. *Stage 2: Building a Service-Oriented Architecture*. Search Engine Project, 2026–2027.

[2] Query Tarantino. *Search Engine Project: Stage 1, Building the Data Layer*. Report, 2026.

[3] M. Nottingham, E. Wilde and S. Dalal. *Problem Details for HTTP APIs*. RFC 9457, IETF, 2023.

[4] OpenAPI Initiative. *OpenAPI Specification*, version 3.1.0, 2021.

[5] MongoDB, Inc. *MongoDB 7.0 Manual*: updates with aggregation pipelines, the `$$NOW` variable and `findOneAndUpdate`. `https://www.mongodb.com/docs/v7.0/`.

[6] Hazelcast, Inc. *Hazelcast Platform 5.7 Documentation*: queues, topics, backups and transactions. `https://docs.hazelcast.com/hazelcast/5.7/`.

[7] The Apache Software Foundation. *Apache ActiveMQ Classic 6 Documentation*: persistence and message acknowledgement. `https://activemq.apache.org/components/classic/`.

[8] OpenJDK. *Java Microbenchmark Harness (JMH)*, version 1.37. `https://github.com/openjdk/jmh`.

[9] J. D. C. Little. A proof for the queuing formula $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.

A Reproducibility

Every run lives in `benchmarks/runs/<run_id>/` of the repository, and its `run_id` ends with the commit it ran from. Each run keeps its `manifest.json` (commit and clean tree, exact command, host, Docker resources, JVM flags, every `TARANTINO_*` variable, and replicas and threads per trial), `corpus.json`, `log.txt` with the hypotheses printed before measuring, the logs of every service, the raw data and `summary.csv`. Every cited run comes from a clean tree on the official machine. The campaign that produced them is `20261105-215348-campaign-c1376c7` (`./scripts/bench/all.sh`); its `runs.txt` lists the 187 steps with their exit codes. `20261106-123526-campaign-b4667d8` (`./scripts/bench/all.sh --only broker-latency`) reran the one step that failed.

Table 25: Source of every table and figure of Section 6 and of Table 12.

In this report	Run	Command	Raw data
Table 17; crawler JMH	`20261105-215351-ingest-throughput-c1376c7`	`./scripts/bench/ingest-throughput.sh`	`ingest.csv`, `stats.csv`, `host.csv`, `jmh/crawler.csv`
Figure 4	`20261105-220915-index-throughput-c1376c7`	`./scripts/bench/index-throughput.sh --without-jmh`	`index.csv`, `stats.csv`
Indexer JMH	`20261103-204454-index-throughput-6925d4e`	`./scripts/bench/index-throughput.sh`	`jmh/indexer.json`, `jmh/indexer.csv`
Figure 5, Table 18; query JMH	`20261104-140900-search-latency-8e1c1e8`	`./scripts/bench/search-latency.sh`	`load/<trial>.csv.gz`, `jmh/query.csv`, `explain/`
Tables 12, 19; Figure 6	`20261106-012054-coordination-variants-c1376c7`	`./scripts/bench/coordination-variants.sh`	`coordination.csv`, `visibility.csv`, `stats.csv`, `variants.csv`, `pairs.csv`
Table 20	`20261105-235758-resource-usage-c1376c7`	`./scripts/bench/resource-usage.sh`	`stats.csv`, `host.csv`
Table 21	`20261106-002208-publication-visibility-c1376c7`	`./scripts/bench/publication-visibility.sh`	`visibility.csv`, `coordination.csv`
Figure 7	`20261106-123534-broker-latency-b4667d8`	`./scripts/bench/broker-latency.sh`	`broker.csv`, `load/`
Table 22	The 165 runs `...-recovery-<nn>[-<mode>]-r<N>-c1376c7`, listed in `sources.txt` of `20261106-073908-recovery-summary-c1376c7`	`TARANTINO_VARIANT=<mode> ./scripts/recovery/<nn>-<row>.sh <N>`	`checks.json`, `summary.csv`
Table 23	`20261106-073911-recovery-delay-c1376c7`	`./scripts/bench/recovery-delay.sh <recovery runs>`	`recovery-delay.csv`

Each table and chart of this report takes its numbers from the `summary.csv`, `variants.csv` or `pairs.csv` of the run named in its caption. The rows they use are in `report/data/` and `report/tables/`. The medians of Table 23 come from `recovery-delay.csv`, over the books of every run of each row. To summarize a run again, delete its `summary.csv` and run its `bench` command on the run folder, for example `java -jar bench/target/bench-1.0-SNAPSHOT.jar coordination benchmarks/runs/<run_id>`; `bench charts` redraws its charts. The report itself builds with `tectonic -X compile QueryTarantino_Stage2_Report.tex` from `report/`.

B Endpoint Inventory

Table 26 lists every HTTP operation of `api/openapi.yaml` (`SPEC.md` §6.1). Besides the responses listed, every endpoint answers `500` with Problem Details to an unexpected exception. Every `202` carries a `Location` header to poll, and every `503` carries `Retry-After: 5`, or `1` on the internal routes.

Table 26: Every HTTP operation, with its provider, nginx port, consumer and responses.

Inventory	Provider	nginx	Method and path	Consumer	Responses
Front-end	`query` (7003)	8080	`GET /search`	Front-end	200, 400, 503
Front-end	`query` (7003)	8080	`GET /books/{book_id}`	Front-end	200, 400, 404, 503
Front-end	`query` (7003)	8080	`GET /search/filters`	Front-end	200, 503
Operational	`crawler` (7001)	8081	`POST /ingest/{book_id}`	Load and admin scripts	202, 200, 400, 503
Operational	`crawler` (7001)	8081	`GET /ingest/status/{book_id}`	Whoever got a 202; checks	200, 400, 404, 503
Operational	`crawler` (7001)	8081	`GET /ingest/list`	Checks	200, 503
Operational	`indexer` (7002)	8081	`POST /index/update/{book_id}`	Admin	202, 200, 400, 404, 409, 503
Operational	`indexer` (7002)	8081	`GET /index/status/{book_id}`	Whoever got a 202	200, 400, 404, 503
Operational	`indexer` (7002)	8081	`POST /index/rebuild`	Admin	202, 503
Operational	`indexer` (7002)	8081	`GET /index/status`	Admin, benchmarks	200, 503
Operational	`control` (7004)	8081	`GET /control/status`	Admin, benchmarks	200, 503
Health	Every service	–	`GET /health`	Compose healthchecks	200, 503
Internal	`crawler` (7001)	8082	`POST /internal/download`	`control`, mode `orchestrator`	202, 400, 503
Internal	`indexer` (7002)	8082	`POST /internal/index`	`control`, mode `orchestrator`	202, 400, 503
Internal	Each `query`	–	`POST /internal/snapshot/refresh`	`control`, mode `orchestrator`	202

Table 27: Answer of each state-changing request, by the book's state (`SPEC.md` §6.2). Every response body is the book's state: `book_id`, `status`, `attempts`, `last_error` and `updated_at`.

Request and book state	Response
`POST /ingest`, no document	"Create the state" and enqueue: `202` with `QUEUED`
`POST /ingest`, `FAILED`	"Retry a `FAILED` book": `202` with `QUEUED` and `attempts` back to 0
`POST /ingest`, `QUEUED` or `DOWNLOADING`	`202` with its state, without enqueuing it again
`POST /ingest`, `DOWNLOADED`, `INDEXING` or `PUBLISHED`	`200` with its state
`POST /index/update`, no document	`404 BOOK_NOT_FOUND`
`POST /index/update`, `QUEUED`, `DOWNLOADING` or `FAILED`	`409 NOT_DOWNLOADED`; a `FAILED` book goes forward with `POST /ingest`
`POST /index/update`, `DOWNLOADED`	`202` with its state, and a `BookTask` to `books-to-index`; a duplicate does no harm
`POST /index/update`, `INDEXING`	`202` with its state, without enqueuing
`POST /index/update`, `PUBLISHED`	`200` with its state

C Failure Matrix

Each row of Table 28 has its own script, `scripts/recovery/<nn>-<row>.sh <N>`. The script brings the stack up, causes the failure, restarts what went down, and checks three things: every book ends `PUBLISHED`, `/search` finds it exactly once, and no book is in two published batches of the current generation (`SPEC.md` §8). A crash point stops the service with `Runtime.halt(1)`, without shutdown hooks, the first time it reaches the named point.

Table 28: What each failure leaves behind, who notices and how the system recovers, in mode `hazelcast`. In the other modes, rows 04, 05 and 12 recover by redelivery or a new dispatch, or do not apply (`SPEC.md` §11.5).

Row	What goes down	What is left	Who notices	How it recovers	How it is caused
01	`crawler` while downloading	A `DOWNLOADING` book with a lease	`control`, when it expires	Back to `QUEUED` and to the queue	`after-claim`
02	`crawler` after the datalake write	The book in the datalake, still `DOWNLOADING`	`control`, when it expires	Back to `QUEUED`; the next claim finds it in the datalake	`after-datalake-write`
03	`crawler` after `DOWNLOADED`, before enqueuing	A `DOWNLOADED` book without a message	`control`: empty queue, old `enqueued_at`	Enqueued again	`after-downloaded`
04, 05	`crawler` or `indexer` after taking messages	Books without a message: an `IQueue` has no acknowledgement	`control`: empty queue, old `enqueued_at`	Enqueued again	`after-dequeue`
06	`indexer` while writing a batch	A `WRITING` batch, `INDEXING` books, half-written postings	`control`, when the batch expires	`ABANDONED`; books back to the queue; postings deleted later	`during-batch-write`
07	`indexer` after publishing, before marking	A `PUBLISHED` batch with `INDEXING` books	`control`	Completes the publication	`after-publish`
08	`indexer` after marking, before notifying	A published batch without `BatchPublished`	`query`, on its refresh	Seen within 5 s	`after-mark`
09	`crawler` stopped longer than its lease	A zombie worker that may still write	`control`, when the lease expires	Requeued; the zombie's writes fail on `claim_id`, and its rename only replaces a complete copy	`docker pause` for 10 s
10	`indexer` stopped longer than its lease	A zombie indexer that keeps writing	Its own renewal fails	It stops writing; its publication fails, and `control` deletes its postings, also late ones	`docker pause` for 10 s
11	`control`	At most books in a new state without their message	Its reconciliation, once back	Reconciles from MongoDB and requeues	`docker kill` while seeding
12	Hazelcast	Empty queues	`control`: empty queue, old `enqueued_at`	Requeues every waiting book	`docker restart`
13	MongoDB	No half-done transition: each one is a single write	Healthchecks; services answer `503`	Everything continues from the last state	`docker stop`, 10 s
14	One `query` replica	Nothing: it does not write	nginx	The other replicas serve; it rebuilds its snapshot at startup	`docker kill`

D Configuration

Every service reads its configuration from environment variables through `TarantinoConfig`, which validates them at startup and stops the service naming any invalid one (`SPEC.md` §4.7). Table 29 lists the variables that set ports, addresses, data paths, replicas, threads and timing. Compose passes the same timing values to every service, and `scripts/recovery/recovery.env` shortens them for the recovery tests while keeping the invariants: heartbeat $\leq$ lease / 3, garbage collection and orphan age $\geq$ 10 leases, and requeue $>$ batch timeout.

Table 29: Main configuration variables, their defaults and the recovery-test values.

Variable	Default	Recovery	Purpose
`TARANTINO_PORT`	7001–7004		HTTP port of `crawler`, `indexer`, `query`, `control`
`TARANTINO_HTTP_THREADS`	Javalin's, 8–250		Maximum request threads
`TARANTINO_MONGO_URI`, `TARANTINO_HAZELCAST`	`localhost`		MongoDB and Hazelcast addresses; Compose points them to its services
`TARANTINO_DATALAKE`, `TARANTINO_WORKLOAD`, `TARANTINO_MIRROR`	`datalake`, `workload`, none		Datalake root, workload folder and local Gutenberg mirror
`TARANTINO_CANDIDATES`	`sample_ids.txt`		Books that `control` seeds, or `mirror`
`TARANTINO_PARALLEL_DOWNLOADS`	8		Download workers per `crawler` replica
`TARANTINO_MAX_ATTEMPTS`	3		Downloads of a book before `FAILED`
`TARANTINO_INDEX_BATCH`	100	5	Books per batch
`TARANTINO_BATCH_TIMEOUT_SECONDS`	30	5	Longest wait to close a batch
`TARANTINO_LEASE_SECONDS`	60	6	Duration of a book or batch lease
`TARANTINO_HEARTBEAT_SECONDS`	20	2	How often a lease is renewed
`TARANTINO_RECONCILE_SECONDS`	30	3	How often `control` reconciles
`TARANTINO_REQUEUE_SECONDS`	120	10	Age after which a lost message is requeued
`TARANTINO_GC_SECONDS`	600	60	Wait before deleting abandoned postings and old generations
`TARANTINO_SNAPSHOT_SECONDS`	5	2	Periodic refresh of the `query` snapshot
`TARANTINO_CORS_ORIGIN`	`*`		Origin that `query` allows
`TARANTINO_COORDINATION`	`hazelcast`		Coordination mode
`TARANTINO_SNAPSHOT_ON_PUBLISH`	`true`		Topic listener in `query`
`TARANTINO_ACTIVEMQ_PERSISTENT`	`false`		Persistent delivery and acknowledgement after the job
`TARANTINO_POLL_MILLIS`	200		Sleep after an empty poll, mode `polling`
`TARANTINO_ORCHESTRATOR_TICK_MILLIS`	200		Pause between ticks, mode `orchestrator`
`JAVA_TOOL_OPTIONS`	`-Xmx512m`, exit when out of memory		Fixed heap; a JVM out of memory exits instead of losing a thread